Semantic Segmentation of Spectral LiDAR Point Clouds Based on Neural Architecture Search

Qi Zhang, Yuanxi Peng, Zhiwen Zhang, and Teng Li

***Abstract*— Multispectral light detection and ranging (LiDAR) point clouds, as a new kind of data, can be characterized by high consistency and integrity of spectral information geometric data. which makes it beneficial for land use classification. However, direct classification of multispectral LiDAR data remains challenging, since the data are irregular and unordered. By describing the point cloud in the form of graph data and using graph convolution to naturally model local geometric structures' representation, higher processing accuracy and efficiency can be achieved. Notably, however, due to the complexity of graph data, using only a single graph convolution to learn the data features may limit the capacity of the architecture; in addition, selecting appropriate graph convolutions requires human expertise and massive numbers of trials. In this article, we propose a network that combines several outstanding graph convolutions by adapting neural architecture search (NAS) for point cloud segmentation, an approach referred to as GCNAS, which can adaptively achieve different levels of semantic expression. In the search module, we use the Monte Carlo tree search (MCTS) method to select the layerwise graph convolution kernels. Experiments on the Titan multispectral LiDAR dataset have verified the effectiveness of GCNAS.**

***Index Terms*— Automated machine learning (AutoML), graph convolution, multispectral light detection and ranging (LiDAR), neural architecture search (NAS), semantic segmentation.**

I. Introduction

LIGHT detection and ranging (LiDAR) offers the great benefit of accurately collecting 3-D information with extreme precision and has accordingly had a significant impact on power line identification, topography mapping, city planning, and environmental surveillance tasks [1], [2]. However, the results of fine-grained point cloud classification are sometimes not ideal due to some inherent characteristics of the LiDAR data. In recent years, multispectral LiDAR systems have been used in the remote sensing field, among others [3], [4]. Compared with single-channel LiDAR data, multispectral LiDAR point clouds have abundant multiwavelength spectral attribute features and have been shown to achieve better classification of finer details [4], [5]. Therefore, multispectral LiDAR point cloud data have been used extensively in classification research. For these 3-D data, semantic segmentation aims to precisely distinguish each point and to increase human comprehension of the 3-D scene. However, due to the unorganized and irregular structure of the data, the effective semantic segmentation of multispectral LiDAR point clouds remains challenging [6].

In previous years, the majority of research on this topic centered around using traditional machine learning approaches to classify multispectral LiDAR point cloud data, including [7], [8], and [9]. However, the feature engineering of this type of method is derived from a set of handcrafted features, which usually demands specialized expert knowledge and high time expenditure.

In the past few years, the advent of deep learning technology has drastically altered research approaches across a wide range of tasks, attracting widespread research attention in fields, such as image classification [10], [11] and object detection [12], [13]. Although deep convolution networks exhibit excellent effectiveness in the domain of 2-D machine learning, they can only receive conventional inputs and cannot be used simply for the 3-D data. Before putting point clouds into the convolutional neural architecture, some studies have opted to turn them into images or standard 3-D voxels, including [14], [15], and [16]. However, this alteration procedure causes a decrease in computational efficiency and an increase in memory use; moreover, transformation of the 2-D images leads to the loss of spatial information.

Deep learning is still developing and has supported further advances in convolutional neural networks (CNNs). Several of these have been used for point cloud segmentation tasks, including the groundbreaking efforts of [17]: this work demonstrated the capability to work with raw point clouds without any preprocessing, inspiring many future studies, such as [18], [19], [20], [21], and [22]. Recently, very promising class diagram structures have been explored in point cloud analysis graph CNNs (GCNs) [23], [24], [25], [26], [27]. Point clouds are described as graph data based on the similarities of structural features among points, and 2-D convolution of the generalized image to 3-D data is performed. Compared with point-feature-based methods, these approaches demonstrate a strong ability to understand context features; it has a strong understanding ability of context features and can achieve higher processing accuracy. Notably, while these are

Manuscript received 23 December 2022; revised 25 April 2023; accepted 8 June 2023. Date of publication 9 June 2023; date of current version 23 June 2023. This work was supported in part by the National Key Research and Development Program of China under Grant 2021ZD0140301; in part by the National Natural Science Foundation of China under Grant 91948303-1, Grant 61803375, Grant 12002380, Grant 62106278, Grant 62101575, and Grant 61906210; and in part by the Postgraduate Scientific Research Innovation Project of Hunan Province under Grant QL20210018. *(Corresponding author: Teng Li.)*

Qi Zhang, Yuanxi Peng, and Zhiwen Zhang are with the State Key Laboratory of High-Performance Computing, College of Computer, National University of Defense Technology, Changsha 410073, China (e-mail: zhangqi.zq0512@alumni.nudt.edu.cn; pyx@nudt.edu.cn; zhangzhiwen20@nudt.edu.cn).

Teng Li is with the College of Advanced Interdisciplinary Studies, National University of Defense Technology, Changsha 410073, China, and also with the Beijing Institute for Advanced Study, National University of Defense Technology, Beijing 100020, China (e-mail: liteng09@nudt.edu.cn).

Digital Object Identifier 10.1109/TGRS.2023.3284995

1558-0644 © 2023 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 22:44:55 UTC from IEEE Xplore. Restrictions apply.

all examples of using abundant human experience to design deep networks, they use only a single feature extractor and exhibit an intrinsic limitation on their ability to achieve different levels of semantic expression.

Recently, an increasing number of researchers have focused on discovering neural network structures automatically rather than relying on human expertise and numerous manual design trials. Neural architecture search (NAS) has shown the potential to discover architectures that are superior to manually designed architectures for image classification [28], image segmentation [29], and 2-D object detection [30]. A recent study by Lin et al. [31] proposes a deep graph representation learning method based on poly-convolution feature kernels that perform point cloud classification for NAS, centering predominantly around classifying point cloud objects. However, influenced by the quality of the feature kernels, it may learn the unfitted semantic information of the point cloud. To address this issue, in this work, we focus on using NAS to explore a segmentation architecture that effectively classifies spectral LiDAR point clouds.

Our motivation is the contribution of graph convolutions to different architectures; in previous practice, we found different effects on different datasets, and we endeavor to find a more general NAS framework for point clouds. One of the main challenges in our work pertains to the generalized expression of various graph convolution operators, particularly in terms of their ability to capture the rich semantic features present in spectral LiDAR data, and another challenge is determining the optimal combination of these operators that would allow architectures to effectively address these features. This article accordingly proposes GCNAS, a novel approach that combines the benefits of graph convolution and NAS for spectral LiDAR point clouds segmentation. Specifically, we design a search space that is composed of multiple graph convolution operations, and then we explore their combination to capture deeper semantic expressions using NAS. During this procedure, GCNAS discovers the architecture using Monte Carlo tree search (MCTS). Our work has the following key elements.

1) We propose a 3-D point cloud segmentation network, called GCNAS, to adaptively capture the feature learning of different levels by combining graph convolution and NAS.
2) We formulate a search space, called GCNAS space, which contains several graph convolutions; based on some advanced work, this search space can be extended using graph-based feature learning.
3) We refer to and use MCTS to investigate the ideal network topology for point clouds' segmentation, which is an effective way to search for high-performing networks.
4) To the best of our knowledge, this article is the first to study NAS for the spectral LiDAR point cloud task.

II. Related Work

A. Traditional Machine Learning Methods

In the early stages of research in this field, handcrafted features of a LiDAR point cloud were studied to distinguish the semantic labels of each point, after which a classifier was selected to identify them. Some works [7], [8] directly classify the data into ten land-ground objects using the support vector machine (SVM) method. The efficacy of using random forest for classifying multispectral LiDAR data was analyzed by Pan et al. [9]. However, previous knowledge of a particular research area is frequently necessary if these techniques are to successfully select the most appropriate classifier [32].

B. Image-Based Deep Learning Methods

Depending on the specific height and level of the intensity data of the multiwavelength 3-D point cloud, these strategies used a rasterizing technique to convert the multispectral LiDAR data into feature images of a specified raster size. They investigated a range of methods for image processing that could enable them to distinguish point clouds and identify objects. Xiaoliang et al. [33] and Matikainen et al. [34] proposed object-level classification methods for the land-cover classification of the Optech Titan Multispectral LiDAR dataset. Bakuła et al. [35] integrated multidimensional data from multispectral LiDAR information to tackle a maximum probability land-cover classification problem. Fernandez-Diaz et al. [36] demonstrated the multispectral LiDAR point clouds' superior performance on land cover, canopy characterization, and bathymetric mapping classification tasks.

C. Point-Based Deep Learning Methods

Methods for point cloud processing are based on multiviews, voxels, graphs, points, etc. Reference [17] is seen as the trailblazer in the field of point utilization. This approach learns the representation of the set of input points in a point-by-point manner through applying a shared multilayer perceptron (MLP) individually. Inspired by Qi et al. [17], recent works [18], [37], [38], [39], [30] have focused on processing the original point cloud. Compared with project-based [40] approaches, point-based approaches can circumvent the detrimental quantization error and artifacts while also eliminating the need for intensive computation and high memory utilization; as a result, such methods are already in widespread use.

Recently, some graph-based works have borrowed ideas from messaging neural networks to effectively use local context and provide a locally dense connection plot that enables them to benefit from the relationships between points in the inputs. The graph-based technique uses points for graph vertices and generates edges in terms of the spatial interactions between the points, which is a natural depiction of the point cloud's regional topology. The concept of graph convolutional networks, first proposed by Kipf and Welling [41], generalizes the convolutional operations on the graph by averaging the features of adjacent nodes. Works proposing similar ideas, such as [25] and [42]. Wang et al. [25], gather the nearest points in feature space and then use the EdgeConv operation to dynamically recognize semantic clues. Inspired by the attention mechanism, Zhang et al. [43] propose a dual attention module that focuses on spatial representation and elevation information to explore long-range dependencies. Landrieu and Simonovsky [27] proposed the superpoint graph (SPG) that

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 22:44:55 UTC from IEEE Xplore. Restrictions apply.

relies on the depiction of the relationships existing among different parts of an object. Lin and Feragen [44] work on neighborhoods of spatial variation and density expansion that can be further adapted by learning the masking attention mechanism. Zhou et al. [26] propose to adaptively establish relationships between points based on the learning characteristics of a pair of points, the adaptability of which represents the diversity of the kernel with respect to each point. These techniques concentrate on designing a single local feature extractor to build a more powerful model while ignoring the constraint of a single feature extractor for different applications.

D. Neural Architecture Search

NAS is committed to finding the optimum architecture design. The NAS approach comprises three components: search space, search strategy, and performance evaluation [45]. The first to explore [46] proposed the neural structure search problem by generating action sequences that can be decoded into neural structures in a reinforcement learning (RL) manner, while Baker et al. [47] proposed using Q-learning agents to search for optimal model architectures. Similar to [46], the methods in [47] are rewarded with accuracy due to training from scratch. There are numerous MCTS-based NAS works to choose from, including [48], [49], and [50]. The method in [50] uses the accuracy of the real network training and the precision generated by the precision predictor to construct the reward. Other NAS algorithms endeavor to effectively evaluate sampling architectures; another simple approach is to train these architectures from the ground up [46]. To facilitate more efficient model training, the larger models adopt weight inheritance, such as by reusing weights for training models to the greatest extent possible. Many NAS works adapt the one-shot model introduced by Pham et al. [51], which shares weights across all the sampling schemes. Recent NAS research works focused on point cloud classification include [52], which proposes point-voxel convolution, in which 3-D input point data are represented by high-resolution points, while convolution is performed in voxels and large groups of points are applied to low-resolution voxel meshes. Tang et al. [53] introduce sparse point voxel convolution to effectively analyze large 3-D scenes, while Lin et al. [54] focus on finding an efficient feature fusion module using NAS. Yang et al. [55] designed a nested neural network architecture to take advantage of multiscale and pyramid hierarchical features. Lin et al. [31] proposed a deep representation learning method that uses poly-convolutional feature kernels to solve the point cloud classification problem. Notably, very few works to date have used NAS to explore the different local feature-extracting kernels to boost point semantic performance.

III. Methodology

In this section, a thorough description of the design process for GCNAS is provided. First, we exploit local geometric characteristics in the spirit of graph convolution and construct feature extractors accordingly. Then, we use NAS to explore the union of feature extractors. Finally, in terms of search strategy, the MCTS is used to discover an effective architecture.

A. Multiple Graph Convolution Kernels

We represent a point cloud in 3-D space as $\mathcal{X} = \{\mathbf{x}_1, \ldots, \mathbf{x}_i, \ldots, \mathbf{x}_N\} \in \mathbb{R}^{N\times C}$; here, each point $\mathbf{x}_i = \underbrace{\{x_i, y_i, z_i, f_{i(1)}, \ldots, f_{i(d)}\}}_{C}$ contains the 3-D coordinates $p_i = \{x_i, y_i, z_i\}$, and f_i which represents the features, such as color, spectral information, and normal. We compute a directed graph $\mathcal{G}(\mathcal{V}, \mathcal{E})$ based on the k-nearest neighbors (KNNs), where $\mathcal{V} = \{1, \ldots, N\}$ is a group of points (nodes), and $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$ denotes a group of edges. The generalized graph convolutions $g()$ of GCNAS can be formulated as follows:

$$g(\mathcal{X}) = \mathcal{S}(\{\mathcal{K}[\mathbf{x}_i, \mathbf{x}_j - \mathbf{x}_i] | \mathbf{x}_j \in \mathcal{N}_i\}). \tag{1}$$

Here, [,] represents a concatenate operation, while $\mathcal{K}()$ denotes a kernel function to map features from MAX/AVG/MLP. According to the edge relation [25], we select the center point $\mathbf{x}_i$ and its local neighborhood information $\mathbf{x}_j - \mathbf{x}_i$. $\mathcal{N}_i$ denotes the KNN points. $\mathcal{S}$ refers to a similar function as $\mathcal{K}$. For a kernel function Θ: $\mathbb{R}^{k\times 2C} \rightarrow \mathbb{R}^M$, we adopt a set of $g()$ to approximate Θ through a certain number of channels, defined as follows:

$$\Theta(\mathcal{X}) \approx \left[g_1(\mathcal{X}), \ldots, g_M(\mathcal{X})\right] \in \mathbb{R}^M. \tag{2}$$

Put simply, given an input $[\mathbf{x}_i, \mathbf{x}_j - \mathbf{x}_i] \in \mathrm{R}^{k\times 2C}$, we get a new feature $f'_{i(m)} \in \mathrm{R}$ by $g_m()$. One approach is to conduct an MLP operation along the dimension of feature dimension $2C$, followed by an MAX operation along the dimension of neighbor size k; this approach is used in [25], in a more flexible manner [31] that involves performing additional operations along these dimensions. The entire set of $g_m()$ can be defined as the following operations.

1) $\mathcal{O}_{(0)} : [\mathbf{x}_i, \mathbf{x}_j - \mathbf{x}_i] \in \mathbb{R}^{k\times 2C} \xrightarrow{\text{MAX}} \mathbb{R}^{2C} \xrightarrow{\text{MLP}} f'_{i(m)} \in \mathbb{R}$.
2) $\mathcal{O}_{(1)} : [\mathbf{x}_i, \mathbf{x}_j - \mathbf{x}_i] \in \mathbb{R}^{k\times 2C} \xrightarrow{\text{MLP}} \mathbb{R}^{2C} \xrightarrow{\text{MLP}} f'_{i(m)} \in \mathbb{R}$.
3) $\mathcal{O}_{(2)} : [\mathbf{x}_i, \mathbf{x}_j - \mathbf{x}_i] \in \mathbb{R}^{k\times 2C} \xrightarrow{\text{MLP}} \mathbb{R}^{k} \xrightarrow{\text{MLP}} f'_{i(m)} \in \mathbb{R}$.
4) $\mathcal{O}_{(3)} : [\mathbf{x}_i, \mathbf{x}_j - \mathbf{x}_i] \in \mathbb{R}^{k\times 2C} \xrightarrow{\text{MLP}} \mathbb{R}^{2C} \xrightarrow{\text{MAX}} f'_{i(m)} \in \mathbb{R}$.
5) $\mathcal{O}_{(4)} : [\mathbf{x}_i, \mathbf{x}_j - \mathbf{x}_i] \in \mathbb{R}^{k\times 2C} \xrightarrow{\text{MLP}} \mathbb{R}^{2C} \xrightarrow{\text{AVG}} f'_{i(m)} \in \mathbb{R}$.
6) $\mathcal{O}_{(5)} : [\mathbf{x}_i, \mathbf{x}_j - \mathbf{x}_i] \in \mathbb{R}^{k\times 2C} \xrightarrow{\text{MLP}} \mathbb{R}^{k} \xrightarrow{\text{AVG}} f'_{i(m)} \in \mathbb{R}$.
7) $\mathcal{O}_{(6)} : [\mathbf{x}_i, \mathbf{x}_j - \mathbf{x}_i] \in \mathbb{R}^{k\times 2C} \xrightarrow{\text{MAX}} \mathbb{R}^{k} \xrightarrow{\text{MLP}} f'_{i(m)} \in \mathbb{R}$.
8) $\mathcal{O}_{(7)} : [\mathbf{x}_i, \mathbf{x}_j - \mathbf{x}_i] \in \mathbb{R}^{k\times 2C} \xrightarrow{\text{MLP}} \mathbb{R}^{k} \xrightarrow{\text{MAX}} f'_{i(m)} \in \mathbb{R}$.
9) $\mathcal{O}_{(8)} : [\mathbf{x}_i, \mathbf{x}_j - \mathbf{x}_i] \in \mathbb{R}^{k\times 2C} \rightarrow \mathbb{R}^{2kC} \xrightarrow{\text{MLP}} f'_{i(m)} \in \mathbb{R}$.
10) $\mathcal{O}_{(9)} : [\mathbf{x}_i, \mathbf{x}_j - \mathbf{x}_i] \in \mathbb{R}^{k\times 2C} \xrightarrow{\text{MLP}} \mathbb{R}^{k\times 3} \xrightarrow{\text{projected sum}} \mathbb{R}^{k} \xrightarrow{\text{MAX}} f'_{i(m)} \in \mathbb{R}$.
11) $\mathcal{O}_{(10)} : [f_i, f_j - f_i] \in \mathbb{R}^{k\times 2d} \xrightarrow{\text{MLP}} \mathbb{R}^{2d} \xrightarrow{*[p_i, p_j - p_i]} \mathbb{R}^{k} \xrightarrow{\text{MAX}} f'_{i(m)} \in \mathbb{R}$.

Here, $*$ denotes the dot product. The last convolution kernel is adopted by adaptive graph convolution (AGConv) [26]. We visualize two typical operations of GCNAS in Fig. 1.

B. Searching Efficient Network Architecture

Despite the flexible kernels, the development of an effective model remains complex and needs to be skillfully optimized. Accordingly, three primitives should be considered in the feature encoding procedure: 1) feature encoding module of the

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 22:44:55 UTC from IEEE Xplore. Restrictions apply.

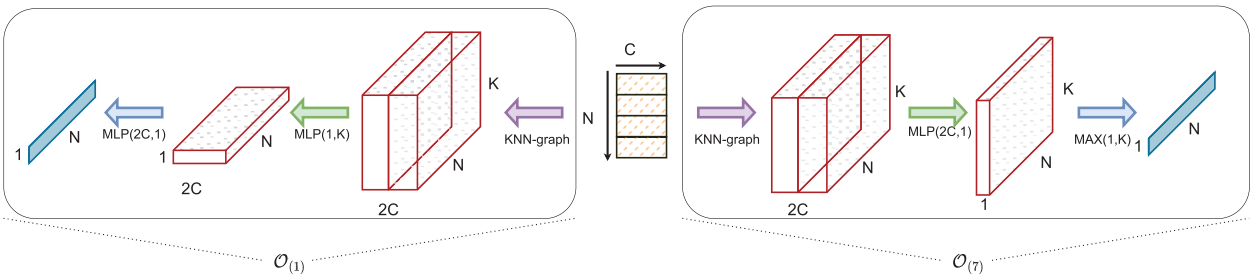


Fig. 1. Visualization of two operations [$\mathcal{O}_{(1)}$ and $\mathcal{O}_{(7)}$] belonging to GCNAS. For the same feature input in the middle, the two feature kernels can extract various features (two groups of new features on the left and right).

input; 2) attention-style module; and 3) residual connection. These should be set based on the specific requirements (for example, accuracy and complexity). In this section, we turn to NAS to automatically design high-performance architecture, motivated by automated machine learning (AutoML). We first endeavor to design a complete search space. Subsequently, we examine the top candidates using MCTS. Finally, we evaluate its performance and train within a single hypernetwork.

1) Search Space: The quality of the search space significantly affects the effectiveness of the architecture design. In our search space, we incorporate 1), 2), and 3) as components of GCNAS inspired by Lin et al. [31].

Consider an encoder–decoder network, which includes an L-layer feature encoder and a decoder with numerous fully connected and pooling layers. Regarding each encoder layer, we denote 1) as $\mathcal{F}(X_{\text{in}})$, where $\mathcal{F}()$ is selected from a type kernel of GCNAS candidates [i.e., $\mathcal{O}_{(0)} - \mathcal{O}_{(10)}$], while (X_{in}) is the input that is obtained from the output of the previous layer, and 2) is carried out by a function $\mathcal{A}()$ that processes the original 3-D coordinates $\mathcal{P}$ and masks the extracted feature $\mathcal{F}(X_{\text{in}})$, i.e., $\mathcal{A}(\mathcal{P}) \odot \mathcal{F}(X_{\text{in}})$. Here, the selection of function $\mathcal{A}()$ and $\mathcal{F}()$ uses the same candidates [except for $\mathcal{O}_{(10)}$ because its implementation consumes a lot of memory during convolution with high-dimensional features], the function of $\mathcal{A}()$ is to establish an alignment between the feature graph of the coordinate space and that of the feature space through $\mathcal{O}$, and this alignment allows for the realization of the attention mechanism through a dot product with the feature graph. We initially encode each potential architecture into a sequence of length $2L$, and L pertains to the count of layers that compose the encoder, such that each element corresponds to an operation. Each of the two adjacent elements in turn is a group that represents a layer of the encoder. Here, the first element is $\mathcal{O}$, while the last element not only determines the $\mathcal{O}$ but also determines whether the outputs of both are concatenated with the input residuals $\mathcal{B}X_{\text{in}}$ (where $\mathcal{B}$ is a binary token that represents the presence or absence of a residual token belonging to the layer). Finally, this search space can be defined as follows:

$$X_{\text{out}} = \mathcal{A}(\mathcal{P}) \odot \mathcal{F}(X_{\text{in}}) + \mathcal{B}X_{\text{in}}. \tag{3}$$

This coding process is shown in Fig. 2.

2) Search Algorithm: The proposed search space has a hierarchical relationship; therefore, it is formulated as a tree structure in which the configuration of previous layers has an impact on subsequent layers. Accordingly, MCTS is adopted,

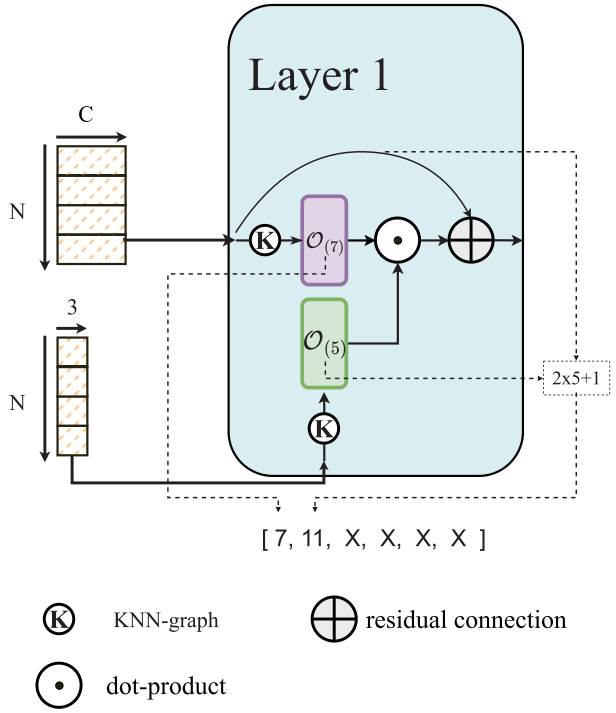


Fig. 2. Illustration of encoding the architecture with two elements for point cloud segmentation based on the feature encoding module $\mathcal{F}()$, attention $\mathcal{A}()$, and residual connection $\mathcal{B}$ for the first layer. The top-left and bottom-left areas of the figure correspond to X_{in} and $\mathcal{P}$, respectively; two graphs were constructed using the KNN method.

which gives us the advantage of making the search more controllable by balancing exploration and exploitation.

The MCTS algorithm needs to establish a tree structure that is composed of nodes representing states. Each node tracks its accumulated reward q and visited time N, as well as the process of transiting itself to the current state. The ith value in the GCNAS search space frame matches the ith layer of the tree. The MCTS search process is similar to the training process for neural networks, in that it eventually stabilizes through constant iteration. At each iteration, three steps are executed as follows.

1) *Selection:* The search path starts from the root node and stops when a leaf node is encountered. Meanwhile, the architectures are encoded by tracing the path. To obtain the current most promising search path, the action of the current node is selected according to Upper Confidence bound applied to the Trees (UCT) [56]

$$\pi_{\text{tree}}(s) = \arg\max_{a} \left(\frac{Q(s,a)}{N(s,a)} + c\sqrt{\frac{\ln(N(s))}{N(s,a)}} \right) \tag{4}$$

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 22:44:55 UTC from IEEE Xplore. Restrictions apply.

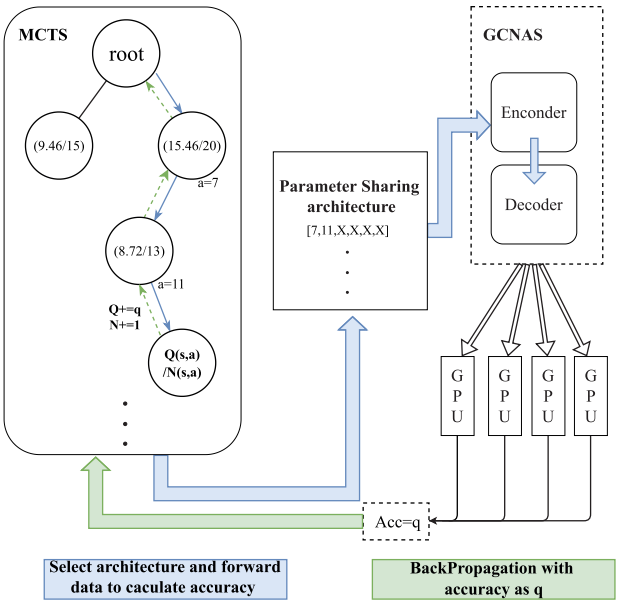


Fig. 3. Process of selecting architecture and forwarding data (blue arrow), and backpropagation (green arrow), with accuracy as reward q and parameter sharing architecture.

where $N(s)$ denotes the number of visits to state s, c is a constant, $N(s,a)$ is the number of times action a is taken at state s, and $Q(s,a)$ is the reward of this action.

2) *Expansion* Add a new node for the tree, if the node is not a leaf and has not been visited. For the new node, the UCT is also used. $Q(s,a)$ and $N(s,a)$ are initialized as follows:

$$N(s,a) = 1$$
$$Q(s,a) = (0.9 + 0.2u)\frac{Q(s)}{N(s)}. \tag{5}$$

Both of them will be updated in the backpropagation step. Here, u is drawn from the interval between 0 and 1, which is often used in Monte Carlo simulations.

3) *Backpropagation:* When looking for a specific node in the tree structure, the route taken to reach it begins at the leaf node and ends at the root. Once the selection is complete, return the architecture and calculate reward q for the new node. That is, update the path of the choice of each state, and update $Q(s,a)$ and $N(s,a)$, and $Q(s,a) = Q(s,a) + q$, $N(s,a) = N(s,a) + 1$; reward q serves as the estimate of the MCTS sampling architecture performance.

3) Evaluate and Train: After each iteration of the search, in theory, the architecture can be trained from scratch, with the final classification accuracy as the reward; however, this results in a high level of computational complexity. Thus, our approach treats the precision of the shared weight architecture as the reward q of the MCTS, similar to the approach used in [51]. After each architecture selection, the model propagates data to obtain the overall accuracy (OA) as the reward. The architecture from the weight-shared model updates parameters through gradient-based backpropagation, after which the architecture is again searched according to the cumulative reward. In summary, the architecture and parameters are alternately updated. The procedure is illustrated in Fig. 3.

After sequentially selecting three consecutive layers of feature encoder, a decoder that contains pooling and linear layers is constructed. Following [25], the outputs from every layer of the encoder are concatenated and then sent to the MLP. Subsequently, the maximum pooling operation is used to derive the global feature; sequentially, the global feature is repeated the same as the input points to concatenate with the output from the encoder as aforementioned, then forwarded through three shared fully connected layers (512, 256, and p), and finally a p-dimensional prediction vector is produced; here, p represents the quantity of classes. Fig. 4 illustrates the configuration of the architecture.

IV. Experiment and Analyses

In this section, we examine the performance of the proposed approach on a large outdoor dataset. Meanwhile, a variety of ablation studies were performed to access the proposed search space and search strategy from different perspectives. We also further compare the proposed GCNAS with other representative methods on Titan multispectral LiDAR data classification.

A. Dataset

Data for the experiment were gathered using the Titan Airborne multispectral LiDAR system [21]. The research area is situated in Whitchurch-Stouffville, a small village (ON, Canada); its center location is located at coordinates 43°58′00″ and 79°15′00″, correspondingly. The initial information included wavelength-specific intensity measurements and geographical coordinates. The new data contained six dimensions (3-D spatial coordinates, three wavelengths, and intensity values). Refer to the original squeeze-and-excitation (SE) block with an improved PointNet++ semantic segmentation network (SE-PointNet++) [21] for more details of the original dataset collection. We study a total of 13 areas, which contain six semantic categories; the semantic categories within each study area and the number of each category can be seen in Fig. 5.

B. Data Preprocessing

When dealing with pretraining data, using a strategy similar to that used in [17], we partition an 80 × 80 m block at random from the training sample. A total of 4096 samples were randomly selected for each block; when the number of samples is less than 4096, we perform a random repeat. Each point is represented as a 9-D vector (XYZ, three wavelengths and intensity values, and normalized spatial coordinates). At the same time, to ensure correct convergence to the optimal solution, we normalize the coordinates of each point. Finally, we perform data augmentation by randomly rotating and jittering the point clouds along the z-axis.

C. Experimental Details

1) Network Configuration: Our network is constructed on the PyTorch framework. All the experiments are implemented on NVIDIA TITAN V with 12-GB memory, the batch size is 32, the network architecture search needs 2000 epochs, and

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 22:44:55 UTC from IEEE Xplore. Restrictions apply.

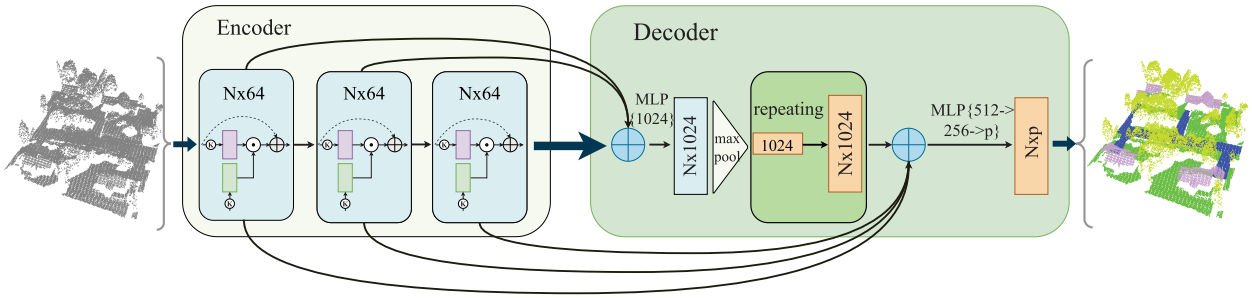
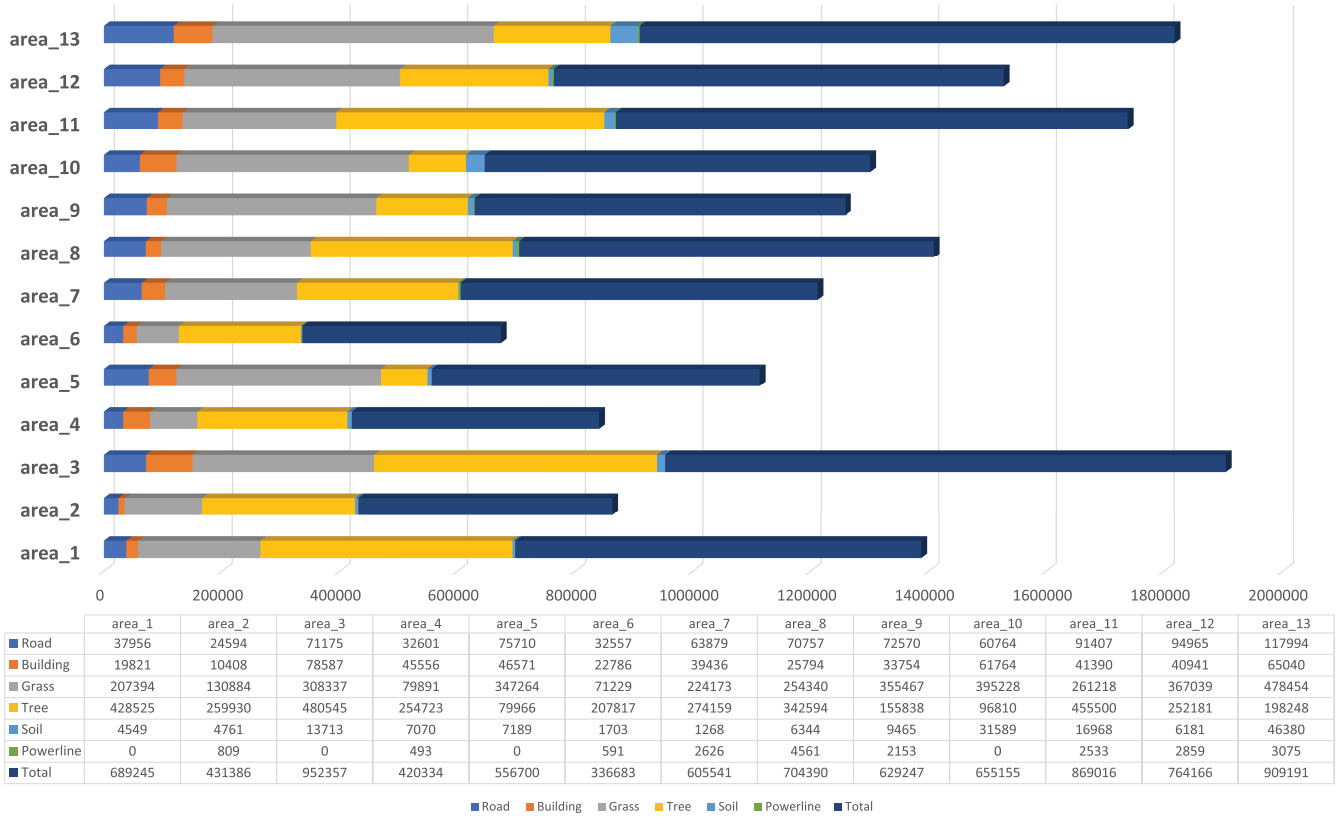


Fig. 4. Fundamental architecture of GCNAS, whose encoder is made up of three components 1), 2), and 3) and refined via NAS.



	area_1	area_2	area_3	area_4	area_5	area_6	area_7	area_8	area_9	area_10	area_11	area_12	area_13
Road	37956	24594	71175	32601	75710	32557	63879	70757	72570	60764	91407	94965	117994
Building	19821	10408	78587	45556	46571	22786	39436	25794	33754	61764	41390	40941	65040
Grass	207394	130884	308337	79891	347264	71229	224173	254340	355467	395228	261218	367039	478454
Tree	428525	259930	480545	254723	79966	207817	274159	342594	155838	96810	455500	252181	198248
Soil	4549	4761	13713	7070	7189	1703	1268	6344	9465	31589	16968	6181	46380
Powerline	0	809	0	493	0	591	2626	4561	2153	0	2533	2859	3075
Total	689245	431386	952357	420334	556700	336683	605541	704390	629247	655155	869016	764166	909191

Fig. 5. Overview of the complete dataset in terms of the number of samples provided; the figures in the table represent the amount of samples.

training the network from scratch also requires 2000 epochs, which is equal to two GPU days. To eliminate the effect of randomness, random seeds were fixed in all the experiments. For the segmentation task, we use stochastic gradient descent (SGD) as the optimizer; the learning rate is set at 0.001, and we use cosine annealing to reduce the decay policy. KNN has been configured to consider 20 neighbor points. Cross-entropy loss helps reduce the gap between what is predicted and the actual labels.

2) Evaluation Index: To gauge the validity of the classification, several evaluation metrics are used. OA is used to measure the proportion of correctly classified examples out of the total population. The Kappa coefficient (Kappa) considers the agreement between the actual and predicted labels, taking the possibility of the agreement occurring by chance into account. Precision and recall are used to measure the exactness and completeness of the classification, respectively. The F1 score is the harmonic mean of precision and recall, while the intersection over union (IoU) is the ratio of the area of overlap between the predicted and actual labels to the total area of the two labels. The formulas are provided below

$$\mathrm{OA} = \frac{\mathrm{TP}+\mathrm{TN}}{\mathrm{TP}+\mathrm{TN}+\mathrm{FP}+\mathrm{FN}} \tag{6}$$

$$\mathrm{IoU} = \frac{\mathrm{TP}}{\mathrm{TP}+\mathrm{FP}+\mathrm{FN}} \tag{7}$$

$$P_e = \frac{(\mathrm{TP}+\mathrm{FP})(\mathrm{TP}+\mathrm{FN})+(\mathrm{TN}+\mathrm{FN})(\mathrm{FP}+\mathrm{TN})}{(\mathrm{TP}+\mathrm{TN}+\mathrm{FP}+\mathrm{FN})^2} \tag{8}$$

$$\mathrm{Kappa} = \frac{\mathrm{OA}-P_e}{1-P_e} \tag{9}$$

$$\mathrm{precision} = \frac{\mathrm{TP}}{\mathrm{TP}+\mathrm{FP}} \tag{10}$$

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 22:44:55 UTC from IEEE Xplore. Restrictions apply.

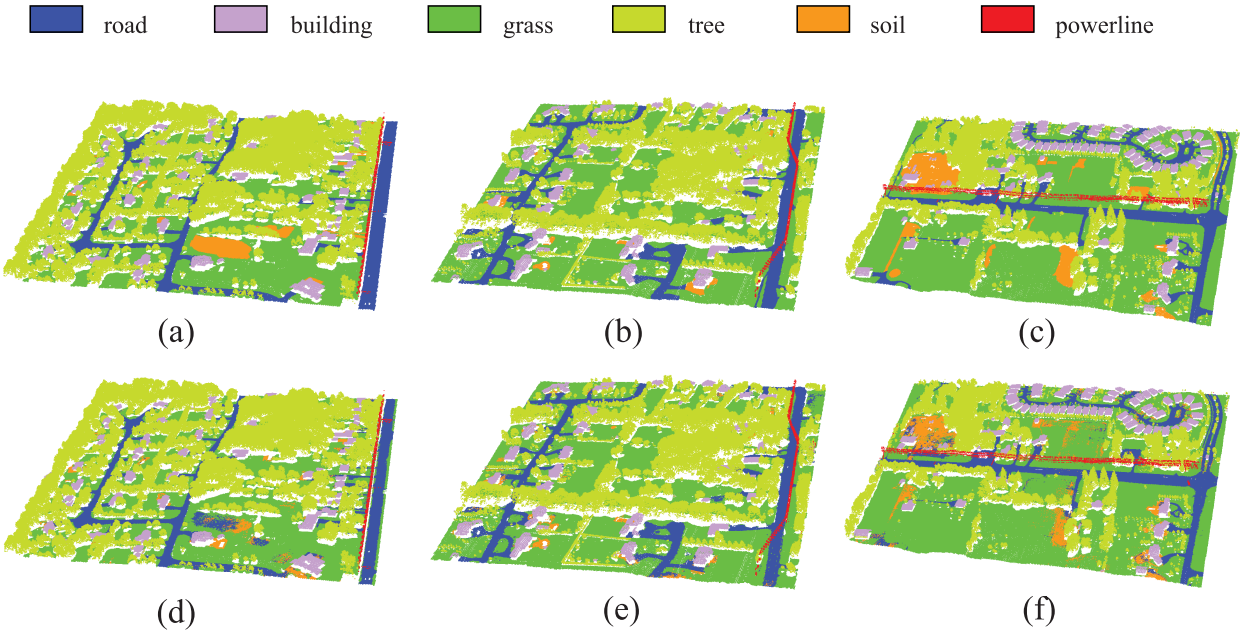


Fig. 6. Our GCNAS classified the test areas (11–13) and represented the results with colors. (a)–(c) Ground truths for areas 11–13, respectively. (d)–(f) Corresponding prediction results.

$$\text{recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \tag{11}$$

$$\text{F1} = 2 * \frac{\text{precision} * \text{recall}}{\text{precision} + \text{recall}}. \tag{12}$$

D. Classification Results

To qualitatively evaluate the classification performance of the proposed model, we present the experimental results in Fig. 6. As shown in Fig. 6, numerous points appear to be accurately classified; the main misclassification points could be found in areas where soil and roads were in proximity. Notably, our method achieves acceptable classification performance on buildings and trees, even if they are of similar height; in addition, it can precisely differentiate both grass and trees, even though they miss obvious borders and have colors on the surface that are similar.

Furthermore, the classification results are further presented in the form of a confusion matrix to investigate further quantitative characterization of the model's classification ability (see Table I), which indicates an F1 score of over 85% for five of the categories. The categories with higher accuracy are trees, grass, buildings, power lines, and roads. Notably, the proposed approach produces remarkable results for tall vegetation, especially trees, which often have serious overlap problems. Even with such a modest number of power lines, our methodology still achieves excellent accuracy. However, our approach is not very powerful for the road category. In more detail, the confusion matrix indicates that many instances of soil are incorrectly identified as roads, most likely because these two categories have comparable geometrical structures and geographic distributions. Furthermore, it is significant to note that the dataset contains a broad range of point clouds for each class, which impedes the model's ability to learn the characteristics of the data; when compared with categories with fewer class labels, classification performance is clearly improved on categories with more class labels. The number of samples differs significantly between categories, as seen in Table I.

E. Ablation Study and Analysis

1) Sensitivities of Parameters: Because our convolution operation depends on KNN graph, it is important to select a suitable number of neighbors K. We accordingly examine the impact of various neighborhood sizes on classification scores through experiments. The K values selected are 8, 20, 32, and 48. In Table II, it is proven that our approach gets the finest benefits while K equals 20. The average F1 scores drop as the size of the built depth graph increases; this has an adverse influence on the classifying results, since it may mean that too many neighbor features are learned.

2) Search Space: To investigate the performance of each operation from our designed search space, as with feature encoders in previous models, we use only a single type of convolutional operator to evaluate the classification performance. Table III shows the results.

As can be observed from Table III, the performance of these graph convolutions is uneven: some achieve preferable performance, while others are not competent for semantic segmentation. This verifies that the manual design of neural networks requires a large number of experiments. Equally unsurprisingly, the proposed GCNAS achieves the best performance, thanks to its ideal composition of several graph convolutions, which proves that our assumption is reasonable. However, we are unable to evaluate additional promising graph convolutions to further increase the search space due to the excessive GPU memory use during the search stage, which constrains the number of convolutional operators.

To further investigate the effectiveness of the designed search space, two similar search spaces are compared: the first

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 22:44:55 UTC from IEEE Xplore. Restrictions apply.

TABLE I

CONFUSION MATRIX OF THE PROPOSED GCNAS. THE NUMBERS IN THE SECOND THROUGH SIXTH LINES REPRESENT THE NUMBER OF POINTS, WHILE THE NUMBERS (%) IN THE LAST THREE ROWS REPRESENT THE PRECISION, RECALL, AND F1 FOR EACH CLASS

Categories	road	building	grass	tree	soil	powerline
road	270265	1	23311	3	10786	0
building	17	145862	39	1188	262	3
grass	22200	491	1059559	3409	21052	0
tree	76	1905	1951	901743	32	222
soil	19095	0	15859	14	34561	0
powerline	0	1	0	423	0	8043
Precision	86.72	98.38	96.26	99.44	51.82	97.28
Recall	88.80	98.98	95.74	99.54	49.71	94.99
F1	87.75	98.68	96.00	99.49	50.74	96.12

TABLE II

RESULTS OF QUANTITY (%) FOR THE GCNAS MODULE USING VARIOUS NUMBERS OF NEIGHBORS. THE FIGURES IN THE TABLE REPRESENT F1 SCORES

Categories	K=8	K=20	K=32	K=48
road	84.39	87.75	86.72	87.57
building	97.24	98.68	98.63	98.73
grass	95.59	96.00	96.27	95.93
tree	99.14	99.49	99.56	99.54
soil	38.36	50.74	46.61	31.30
powerline	87.62	96.12	97.13	96.10
avg.F1	83.72	88.13	87.49	84.86

TABLE III

RESULTS (%) COMPARISON BETWEEN UNIQUE CONVOLUTION OPERATION AND COMBINATORIAL OPERATIONS WHICH IS HIGHLIGHTED IN THE LAST LINE

Convolution operator	OA(%)	mIoU(%)
$\mathcal{O}_{(0)}$	93.79	75.02
$\mathcal{O}_{(1)}$	93.37	73.07
$\mathcal{O}_{(2)}$	93.34	75.15
$\mathcal{O}_{(3)}$	91.39	62.75
$\mathcal{O}_{(4)}$	89.71	65.85
$\mathcal{O}_{(5)}$	89.25	68.52
$\mathcal{O}_{(6)}$	91.11	60.61
$\mathcal{O}_{(7)}$	93.36	70.71
$\mathcal{O}_{(8)}$	92.39	71.95
$\mathcal{O}_{(9)}$	93.68	74.01
$\mathcal{O}_{(10)}$	92.21	70.12
ours	**95.19**	**82.23**

contains a different combination of PolyConv [31], while the second is our GCNAS, which extends the convolution kernel of AGconv. The classification results are presented in Table IV.

The results show that GCNAS enables the baseline to gain 1.2% and 1.3% in terms of average F1 score and mIoU, respectively, since the first ten operations of PolyConv are the same as ours; the only difference is that we introduce AGConv. Thus, the performance gain of our GCNAS can be attributed to the introduction of AGConv, which improves the quality of the search space; this is because AGConv is more helpful to capture the different relationship between different semantic parts.

TABLE IV

RESULTS (%) COMPARISON BETWEEN POLYCONV AND OUR SEARCH SPACE UNDER THREE TEST AREAS (11–13)

Categories	PolyConv [31]	GCNAS
road	86.8	87.7
building	98.3	98.6
grass	96.1	96.0
tree	99.3	99.4
soil	48.6	50.7
powerline	94.1	96.1
avg.F1	86.9	88.1
mIoU	80.9	82.2

TABLE V

RESULTS (%) COMPARISON BETWEEN THE MCTS AND THE RANDOM SEARCH ($c = 10$) UNDER THREE TEST AREAS (11–13). COMPARISON WITH OTHER EXPLORATORY PREFERENCES (DIFFERENT c) IS ALSO PROVIDED

c	0.5	0.8	1	1.5	10
OA(%)	93.13	95.18	95.19	90.17	94.61
mIoU(%)	77.12	80.62	82.23	70.06	78.54

3) Search Strategy: To verify the effectiveness of our proposed search strategy, we choose random search for baseline comparison. Specifically, we compare the MCTS algorithm with random search and implement random search by controlling the parameter c; this is because, when using MCTS, c determines whether exploration is preferred. As shown in Table V, MCTS can achieve the best performance with $c = 1$: under this condition, the model secures an improvement in mIoU of 3.95% compared with the random search. This confirms at least on the multispectral LiDAR dataset, $c = 1$ is conducive to reasonably mining this search space.

4) Robustness: To assess the robustness of our proposed method, information loss under disturbed conditions is simulated through inputting different numbers of points. Specifically, we varied N from 4096 to 2048, 1024, and 512. The results are illustrated in Fig. 7.

As can be readily observed, the performance of our proposed GCNAS is relatively stable and robust under different numbers of input points. Moreover, we found that setting the number of sampling points to 2048 facilitates optimal

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 22:44:55 UTC from IEEE Xplore. Restrictions apply.

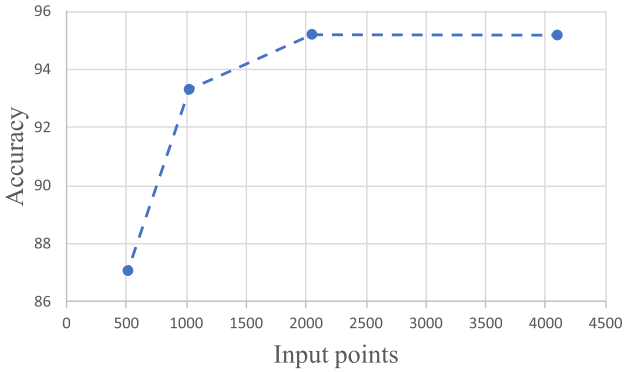


Fig. 7. Illustration of model performance under different sampling group points.

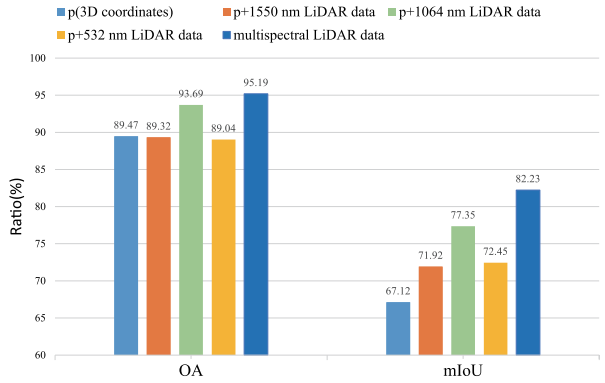


Fig. 8. Results of varied input data for classifying point clouds.

sampling points set to 2048 being more proper for the semantic segmentation of the multispectral LiDAR dataset when considering the computational cost and efficiency.

5) Input Data: To analyze and compare the different spectral information channels of multispectral LiDAR data, we carried out several experiments with five different types of input data: 1) only original 3-D coordinates $\mathcal{P}$ without any intensities; 2) $\mathcal{P}$ with 1550-nm LiDAR data; 3) $\mathcal{P}$ with 1064-nm LiDAR data; 4) $\mathcal{P}$ with 532-nm LiDAR data; and 5) multispectral LiDAR data. In this group of experiments, the other experimental settings were kept the same. The average experimental results for the chosen test area are shown in Fig. 8.

As seen from Fig. 8, the point cloud classification result using LiDAR data that fuse all the spectral channels was the best: the OA improved by 6%, while the mIoU score experienced a remarkable jump from 4% to 14%. The point cloud classification performance of 1064-nm LiDAR data was the second best, whereas the results of the 1550-nm LiDAR data were almost identical to those of the 532-nm LiDAR data.

6) Comparison: Since there are few algorithms applied to multispectral LiDAR point clouds at present, we compare the proposed method with several typical methods, including deep learning on point sets for 3D classification and segmentation (PointNet) [17], graph attention convolution network (GACNet) [24], PointNet++ [18], relation-shape convolutional neural network (RSCNN) [19] SE-PointNet++ [21], SPG [27], efficient semantic segmentation of large-scale point clouds (RandLA-Net) [57], Reference network (ResGCN-28) with 28 layers and MRGCN (Max-Relative GCN) operator

TABLE VI
QUANTITATIVE COMPARISON OF OUR METHOD AND OTHER WELL-KNOWN MODELS. THE HIGHEST EVALUATION SCORE IS SHOWN IN BOLD TYPE

model	OA(%)	mIoU(%)	F1-Score(%)	Kappa
PointNet [17]	83.79	44.28	46.68	0.73
GACNet [24]	89.91	55.14	69.64	0.84
PointNet++ [18]	90.09	58.60	70.13	0.84
DGCNN [25]	93.38	71.42	77.80	0.89
RSCNN [19]	90.99	56.10	70.23	0.86
AGConv [26]	93.27	71.18	76.87	0.89
SE-PointNet++ [21]	91.16	60.15	73.14	0.86
SPG [27]	93.13	61.76	73.28	0.86
RandLA-Net [57]	93.65	74.41	81.16	0.89
ResMRGCN-28 [58]	93.27	73.95	81.06	0.88
GCNAS	**95.19**	**82.23**	**88.13**	**0.93**

(ResMRGCN-28) [58], dynamic graph CNN (DGCNN) [25], and AGConv [26]; the input data for these methods are in the form of multispectral LiDAR data. The results are shown in Table VI.

In terms of each of the evaluation metrics for point cloud classification, PointNet had the poorest performance. This is because PointNet considers only the capture of global features, while ignoring the extraction of local features; this results in information loss, making it unsuitable for fine-grained semantic segmentation tasks. GACNet has slightly better performance, with an OA of 89.91%; because GACNet integrates the attention mechanism and graph convolution, it exhibits an enhanced ability to capture local shapes, which makes it more flexible when learning the structural features of objects, improves the quality of feature representation, and thus helps it to achieve accurate point cloud classification. PointNet++ set abstraction (SA) and multiscale grouping (MSG) for multiscale information fusion, resulting in improved point cloud segmentation performance. RSCNN learns from the relations of the geometric topology constraint among points to obtain discriminative shape awareness. Moreover, DGCNN and AGConv incorporate local neighborhood information and act on dynamically computed graphs, making them closely related to our work. However, our model not only outperformed these other models in terms of average F1 scores but also achieved excellent results more generally; this is because NAS has the advantage of automatically mining the potential of the network. Thanks to the good quality of the search space, which combines the advantages of different graph convolutions and an effective search algorithm that optimizes network details according to the data characteristics, the classification results of our GCNAS are superior. Experiments show that our network has significantly improved segmentation results, improving accuracy by 1.54% and mIoU by 7.82% compared with the state-of-the-art methods, as discussed above.

V. CONCLUSION

In this article, we propose a segmentation network, GCNAS, which consists of multiple graph convolution kernels. These convolution kernels are highly advanced and therefore able to provide an in-depth understanding of contextual features. Aiming to explore their optimal combination and accordingly obtain the most promising architecture in the semantic

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 22:44:55 UTC from IEEE Xplore. Restrictions apply.

segmentation of spectral LiDAR point clouds, the MCTS method is adapted to automatically discover architectures to the greatest extent possible. Moreover, this approach also avoids the trial and error of manually exploring the architecture. In addition, numerous experiments were carried out to verify the effectiveness of GCNAS on the Titan airborne multispectral LiDAR data. Quantitative experimental analysis revealed that GCNAS is effective and robust for semantic segmentation.

ACKNOWLEDGMENT

The authors would like to thank the anonymous reviewers for their valuable suggestions. They would also like to thank Jonathan Li and Haiyan Guan.

REFERENCES

[1] C. Wen, L. Yang, X. Li, L. Peng, and T. Chi, "Directionally constrained fully convolutional neural network for airborne LiDAR point cloud classification," *ISPRS J. Photogramm. Remote Sens.*, vol. 162, pp. 50–62, Apr. 2020.

[2] W. Li, F.-D. Wang, and G.-S. Xia, "A geometry-attentional network for ALS point cloud classification," *ISPRS J. Photogramm. Remote Sens.*, vol. 164, pp. 26–40, Jun. 2020.

[3] N. Yokoya et al., "Open data for global multimodal land use classification: Outcome of the 2017 IEEE GRSS data fusion contest," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 11, no. 5, pp. 1363–1377, May 2018.

[4] Y. Xu et al., "Advanced multi-sensor optical remote sensing for urban land use and land cover classification: Outcome of the 2018 IEEE GRSS data fusion contest," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 12, no. 6, pp. 1709–1724, Jun. 2019.

[5] T.-A. Teo and H.-M. Wu, "Analysis of land cover classification using multi-wavelength LiDAR system," *Appl. Sci.*, vol. 7, no. 7, p. 663, Jun. 2017.

[6] M. Yousefhussien, D. J. Kelbe, E. J. Ientilucci, and C. Salvaggio, "A multi-scale fully convolutional network for semantic labeling of 3D point clouds," *ISPRS J. Photogramm. Remote Sens.*, vol. 143, pp. 191–204, Sep. 2018.

[7] N. Ekhtari, C. Glennie, and J. C. Fernandez-Diaz, "Classification of multispectral LiDAR point clouds," in *Proc. IEEE Int. Geosci. Remote Sens. Symp. (IGARSS)*, Jul. 2017, pp. 2756–2759.

[8] N. Ekhtari, C. Glennie, and J. C. Fernandez-Diaz, "Classification of airborne multispectral LiDAR point clouds for land cover mapping," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 11, no. 6, pp. 2068–2078, Jun. 2018.

[9] S. Pan, H. Guan, Y. Yu, J. Li, and D. Peng, "A comparative land-cover classification feature study of learning algorithms: DBM, PCA, and RF using multispectral LiDAR data," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 12, no. 4, pp. 1314–1326, Apr. 2019.

[10] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional networks for biomedical image segmentation," in *Proc. Int. Conf. Med. Image Comput. Comput.-Assist. Intervent*. Cham, Switzerland: Springer, 2015, pp. 234–241.

[11] J. Long, E. Shelhamer, and T. Darrell, "Fully convolutional networks for semantic segmentation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2015, pp. 3431–3440.

[12] W. Liu et al., "SSD: Single shot multibox detector," in *Proc. Eur. Conf. Comput. Vis.* Cham, Switzerland: Springer, 2016, pp. 21–37.

[13] T. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Oct. 2017, pp. 2999–3007.

[14] Z. Yang, B. Tan, H. Pei, and W. Jiang, "Segmentation and multi-scale convolutional neural network-based classification of airborne laser scanner data," *Sensors*, vol. 18, no. 10, p. 3347, Oct. 2018.

[15] R. Zhao, M. Pang, and J. Wang, "Classifying airborne LiDAR point clouds via deep features learned by a multi-scale convolutional neural network," *Int. J. Geographical Inf. Sci.*, vol. 32, no. 5, pp. 960–979, May 2018.

[16] Y. Ben-Shabat, M. Lindenbaum, and A. Fischer, "3D point cloud classification and segmentation using 3D modified Fisher vector representation for convolutional neural networks," 2017, *arXiv:1711.08241*.

[17] R. Q. Charles, H. Su, M. Kaichun, and L. J. Guibas, "PointNet: Deep learning on point sets for 3D classification and segmentation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 77–85.

[18] C. R. Qi, L. Yi, H. Su, and L. J. Guibas, "PointNet++: Deep hierarchical feature learning on point sets in a metric space," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2017, pp. 5099–5108.

[19] Y. Liu, B. Fan, S. Xiang, and C. Pan, "Relation-shape convolutional neural network for point cloud analysis," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 8887–8896.

[20] H. Thomas, C. R. Qi, J. Deschaud, B. Marcotegui, F. Goulette, and L. Guibas, "KPConv: Flexible and deformable convolution for point clouds," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2019, pp. 6410–6419.

[21] Z. Jing et al., "Multispectral LiDAR point cloud classification using SE-PointNet++," *Remote Sens.*, vol. 13, no. 13, p. 2516, Jun. 2021.

[22] F. Zhang, J. Fang, B. W. Wah, and P. H. S. Torr, "Deep fusionnet for point cloud semantic segmentation," in *Proc. Eur. Conf. Comput. Vis.*, Aug. 2020, pp. 644–663.

[23] Z. Lin, S. Huang, and Y. F. Wang, "Convolution in the cloud: Learning deformable kernels in 3D graph convolution networks for point cloud analysis," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2020, pp. 1797–1806.

[24] L. Wang, Y. Huang, Y. Hou, S. Zhang, and J. Shan, "Graph attention convolution for point cloud semantic segmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 10288–10297.

[25] Y. Wang, Y. Sun, Z. Liu, S. E. Sarma, M. M. Bronstein, and J. M. Solomon, "Dynamic graph CNN for learning on point clouds," *ACM Trans. Graph.*, vol. 38, no. 5, pp. 1–12, Oct. 2019.

[26] H. Zhou, Y. Feng, M. Fang, M. Wei, J. Qin, and T. Lu, "Adaptive graph convolution for point cloud analysis," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2021, pp. 4945–4954.

[27] L. Landrieu and M. Simonovsky, "Large-scale point cloud semantic segmentation with superpoint graphs," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 4558–4567.

[28] B. Zoph, V. Vasudevan, J. Shlens, and Q. V. Le, "Learning transferable architectures for scalable image recognition," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 8697–8710.

[29] L.-C. Chen et al., "Searching for efficient multi-scale architectures for dense image prediction," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 31, 2018, pp. 8699–8710.

[30] A. Xu, A. Yao, A. Li, A. Liang, and A. Zhang, "Auto-FPN: Automatic network architecture adaptation for object detection beyond classification," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2019, pp. 6648–6657.

[31] X. Lin, K. Chen, and K. Jia, "Object point cloud classification via poly-convolutional architecture search," in *Proc. 29th ACM Int. Conf. Multimedia*, Oct. 2021, pp. 807–815.

[32] T. Hackel, J. D. Wegner, and K. Schindler, "Fast semantic segmentation of 3D point clouds with strongly varying density," *ISPRS Ann. Photogramm., Remote Sens. Spatial Inf. Sci.*, vol. 3 pp. 177–184, Jul. 2016.

[33] X. Zou, G. Zhao, J. Li, Y. Yang, and Y. Fang, "3D land cover classification based on multispectral LiDAR point clouds," *Int. Arch. Photogramm., Remote Sens. Spatial Inf. Sci.*, vols. XLI–B1, pp. 741–747, Jun. 2016.

[34] L. Matikainen, K. Karila, J. Hyyppä, P. Litkey, E. Puttonen, and E. Ahokas, "Object-based analysis of multispectral airborne laser scanner data for land cover classification and map updating," *ISPRS J. Photogramm. Remote Sens.*, vol. 128, pp. 298–313, Jun. 2017.

[35] K. Bakuła, P. Kupidura, and Ł. Jełowicki, "Testing of land cover classification from multispectral airborne laser scanning data," *ISPRS Int. Arch. Photogramm., Remote Sens. Spatial Inf. Sci.*, vol. XLI-B7, pp. 161–169, Jun. 2016.

[36] J. Fernandez-Diaz et al., "Capability assessment and performance metrics for the Titan multispectral mapping LiDAR," *Remote Sens.*, vol. 8, no. 11, p. 936, Nov. 2016.

[37] Y. Xu, T. Fan, M. Xu, L. Zeng, and Y. Qiao, "SpiderCNN: Deep learning on point sets with parameterized convolutional filters," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, Sep. 2018, pp. 87–102.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 22:44:55 UTC from IEEE Xplore. Restrictions apply.

[38] W. Wu, Z. Qi, and L. Fuxin, "PointConv: Deep convolutional networks on 3D point clouds," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 9613–9622.
[39] X. Yan, C. Zheng, Z. Li, S. Wang, and S. Cui, "PointASNL: Robust point clouds processing using nonlocal neural networks with adaptive sampling," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2020, pp. 5588–5597.
[40] T. Le and Y. Duan, "PointGrid: A deep network for 3D shape understanding," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 9204–9214.
[41] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," 2016, *arXiv:1609.02907*.
[42] Y. Shen, C. Feng, Y. Yang, and D. Tian, "Mining point cloud local structures by kernel correlation and graph pooling," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 4548–4557.
[43] K. Zhang et al., "A dual attention neural network for airborne LiDAR point cloud semantic segmentation," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5704617.
[44] M. Lin and A. Feragen, "DiffConv: Analyzing irregular point clouds with an irregular view," in *Proc. Eur. Conf. Comput. Vis.* Cham, Switzerland: Springer, 2022, pp. 380–397.
[45] T. Elsken, J. H. Metzen, and F. Hutter, "Neural architecture search: A survey," *J. Mach. Learn. Res.*, vol. 20, no. 1, pp. 1997–2017, 2019.
[46] B. Zoph and Q. V. Le, "Neural architecture search with reinforcement learning," 2016, *arXiv:1611.01578*.
[47] B. Baker, O. Gupta, N. Naik, and R. Raskar, "Designing neural network architectures using reinforcement learning," 2016, *arXiv:1611.02167*.
[48] R. Negrinho and G. Gordon, "DeepArchitect: Automatically designing and training deep architectures," 2017, *arXiv:1704.08792*.
[49] L. Wang, Y. Zhao, Y. Jinnai, Y. Tian, and R. Fonseca, "Neural architecture search using deep neural networks and Monte Carlo tree search," 2018, *arXiv:1805.07440*.
[50] M. Wistuba, "Finding competitive network architectures within a day using UCT," 2017, *arXiv:1712.07420*.
[51] H. Pham, M. Guan, B. Zoph, Q. Le, and J. Dean, "Efficient neural architecture search via parameters sharing," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 4095–4104.
[52] Z. Liu, H. Tang, Y. Lin, and S. Han, "Point-voxel CNN for efficient 3D deep learning," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 32, 2019.
[53] H. Tang et al., "Searching efficient 3D architectures with sparse point-voxel convolution," in *Proc. Eur. Conf. Comput. Vis.* Cham, Switzerland: Springer, 2020, pp. 685–702.
[54] H. Lin et al., "Semantic segmentation of 3D indoor LiDAR point clouds through feature pyramid architecture search," *ISPRS J. Photogramm. Remote Sens.*, vol. 177, pp. 279–290, Jul. 2021.
[55] F. Yang, X. Li, and J. Shen, "Nested architecture search for point cloud semantic segmentation," *IEEE Trans. Image Process.*, vol. 32, pp. 2889–2900, 2022.
[56] L. Kocsis and C. Szepesvári, "Bandit based monte-carlo planning," in *Proc. Eur. Conf. Mach. Learn.* Cham, Switzerland: Springer, 2006, pp. 282–293.
[57] Q. Hu et al., "RandLA-Net: Efficient semantic segmentation of large-scale point clouds," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2020, pp. 11105–11114.
[58] G. Li et al., "DeepGCNs: Making GCNs go as deep as CNNs," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 6, pp. 6923–6939, Jun. 2023.



Qi Zhang is currently pursuing the master's degree with the State Key Laboratory of High-Performance Computing, College of Computer, National University of Defense Technology, Changsha, China.

His research interests include deep learning, point cloud analysis, and intelligent perception.



Yuanxi Peng was born in 1966. He received the B.S. degree from Sichuan University, Chengdu, China, in 1988, and the M.S. and Ph.D. degrees from the National University of Defense and Technology (NUDT), Changsha, China, in 1998 and 2001, respectively, all in computer science.

He was a Visiting Professor with the Department of Electronic and Computer Engineering, University of Toronto, Toronto, ON, Canada, from 2010 to 2011. He has been a Professor with the Computer School, NUDT, since 2011. His research interests include hyperspectral image processing, high-performance computing, multicore and many-core architectures, on-chip networks, cache coherence protocols, and architectural support for parallel programming.



Zhiwen Zhang received the B.Sc. degree in information and computing science from Shandong University, Jinan, China, in 2020. He is currently pursuing the M.S. degree in electronic science and technology with the State Key Laboratory of High-Performance Computing, National University of Defense Technology, Changsha, China.

His research interests include hyperspectral image processing, point cloud analysis, and intelligent perception.



Teng Li received the B.S., M.S., and Ph.D. degrees from the National University of Defense Technology, Changsha, China, in 2013, 2015, and 2020, respectively.

Since January 2021, he has been an Assistant Professor with the Beijing Institute for Advanced Study, College of Advanced Interdisciplinary Studies, National University of Defense Technology, Beijing, China. His research interests include machine learning, hyperspectral image processing, and representation learning.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 22:44:55 UTC from IEEE Xplore. Restrictions apply.